



ESCUELA PROFESIONAL DE INGENIERÍA DE COMPUTACIÓN Y SISTEMAS

**INFORME DE AVANCE — SPRINT 1
DESPLIEGUE CLOUD DEVSECOPS: VERCEL + RENDER**

**PROYECTO:
AGROSMART INSIGHTS – OPTIMIZACIÓN DE MERCADOS Y PRECIOS
AGRÍCOLAS**

**CURSO:
TALLER DE PROYECTOS**

**PROFESOR:
ING. NORMA VIRGINIA LEÓN LESCANO**

**INTEGRANTE:
León Cangalaya, Gabriel Emilio
Líder DevSecOps – Célula 2**

FECHA: 25/08/2026

LIMA – PERÚ

INFORME DE AVANCE SPRINT 1 — DESPLIEGUE CLOUD DEVSECOPS

1. Objetivo

El presente informe tiene por objetivo documentar, sustentar y consolidar las actividades técnicas desarrolladas por el Líder DevSecOps durante las sesiones del 24 y 25 de agosto de 2026 del Sprint 1 en el proyecto AgroSmart Insights. El alcance comprende la integración controlada en main mediante Pull Requests, el despliegue del frontend en Vercel, el aprovisionamiento de n8n y PostgreSQL en Render, la resolución de incidencias de memoria y esquema, la carga de 9 445 registros del GMMML, la activación de WF2 y la primera prueba de extremo a extremo en producción.

1.1. Fundamento metodológico

Se implementó un marco DevSecOps transversal articulado con el flujo de trabajo GitFlow institucional. Cada componente de software se somete a validaciones automáticas de seguridad, formateo estricto y pruebas unitarias previo a su fusión en la rama de producción, dando cumplimiento a las buenas prácticas de desarrollo seguro de software del estándar NIST SP 800-218 (NIST, 2022).

1.2. Historias de usuario atendidas en la sesión

- S1-01 — Desplegar servidor Cloud y unificar pipeline CI/CD en GitHub Actions (8 Story Points, Issue #28).
- S1-06 — Configurar bloqueo estricto de PRs por fallos en pipeline de linters/tests (5 Story Points, Issue #29).

2. Integración y Gobernanza del Repositorio (Pull Requests)

2.1. Integración general del Sprint 1 (Pull Request #34)

Se gestionó la integración del stack completo del Sprint 1 mediante el Pull Request #34 hacia main. Este PR incorporó el frontend React agrosmart-frontend, los flujos de n8n WF0/WF1/WF2, el esquema relacional con pgvector, el pipeline deploy.yml, la especificación render.yaml y los scripts operativos health-check.sh y rollback.sh. Los tres status checks obligatorios (validate-frontend, Auditar JSONs de n8n, Pruebas Unitarias de IA) finalizaron exitosamente y se obtuvo la aprobación del revisor Cristhian0520.

2.2. Hotfix de infraestructura Blueprint (Pull Request #35)

Durante el despliegue inicial en Render se identificó un fallo en la resolución de la imagen Docker de n8n. Se creó la rama fix/render-docker-image-url y se tramitó el PR #35, corrigiendo la URL hacia docker.io/n8nio/n8n:1.83.2 e incorporando N8N_ENCRYPTION_KEY con generación automática. El PR fue validado y fusionado, cerrando automáticamente los issues #28 y #29.

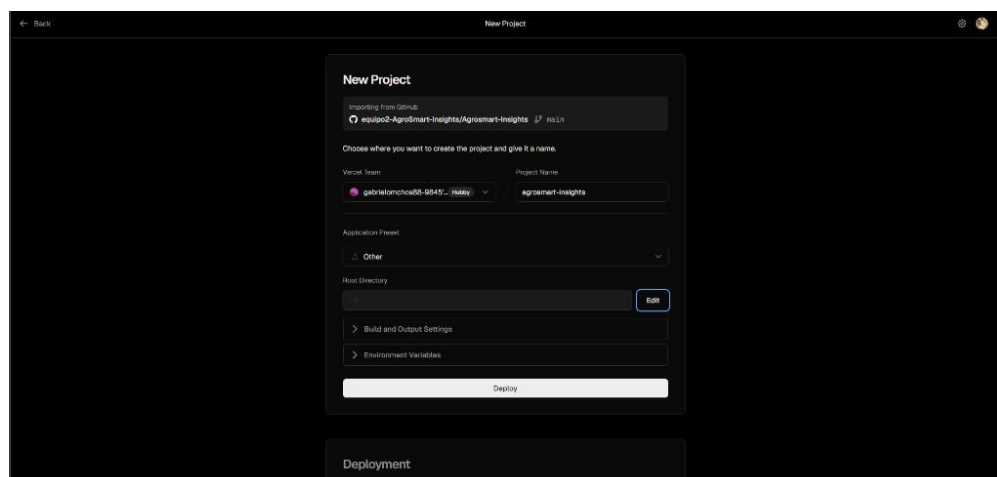
3. Configuración y Despliegue en Vercel (Frontend)

Se completó la puesta en producción del frontend agrosmart-frontend conectando el repositorio a Vercel bajo los siguientes parámetros técnicos:

- Repositorio institucional: equipo2-AgroSmart-Insights/Agrosmart-Insights (rama main).
- Root Directory: src/frontend/agrosmart-frontend (detección automática de Vite).
- Variable de entorno: VITE_N8N_WEBHOOK_URL configurada con <https://agrosmart-n8n.onrender.com/webhook/v1/query>.

Figura 1

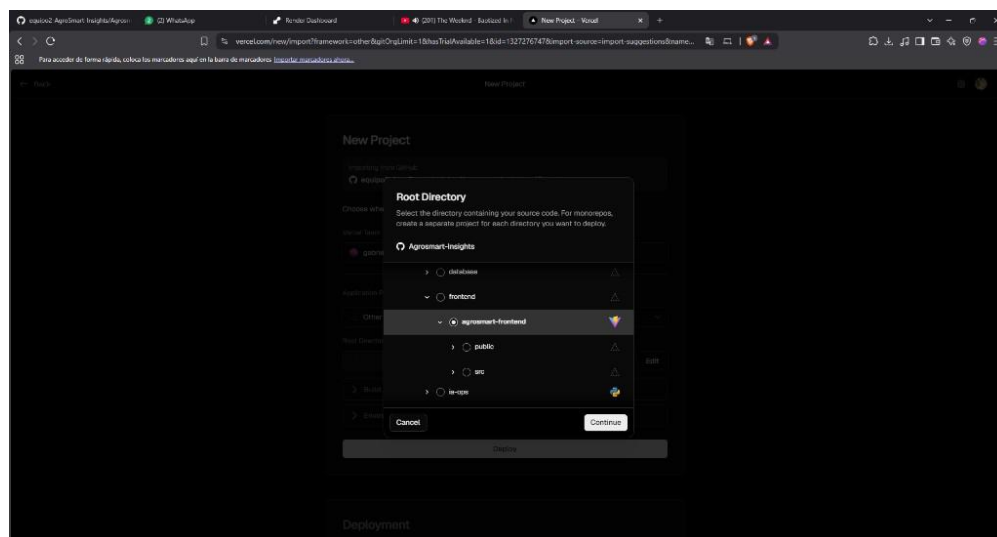
Importación y selección de Root Directory para el Frontend en Vercel



Nota. Captura de pantalla de la interfaz de Vercel vinculando el repositorio institucional y la rama de producción main. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

Figura 2

Selector de Root Directory apuntando a src/frontend/agrosmart-frontend



Nota. Captura de pantalla del selector de directorio en Vercel confirmando la ruta del microfrontend React y el framework Vite. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

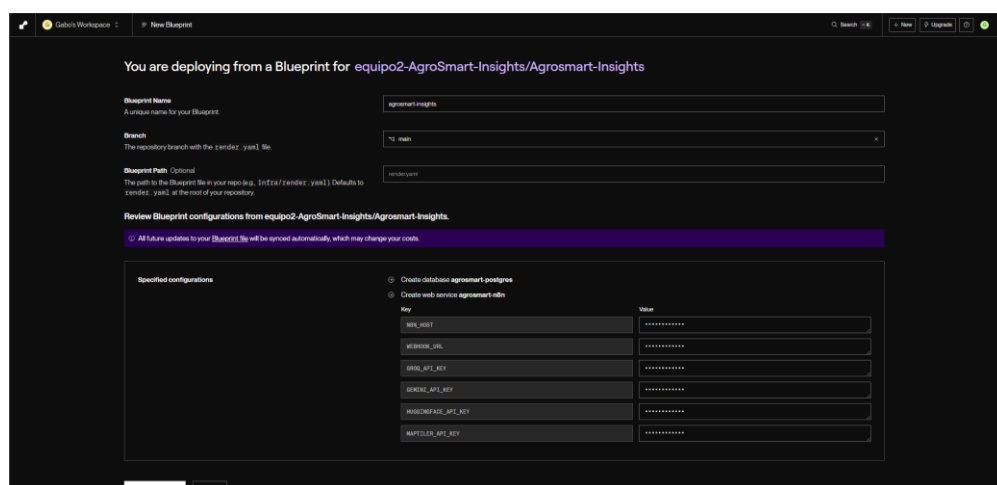
4. Despliegue en Render (n8n + PostgreSQL)

4.1. Creación de Blueprint Instance

Se utilizó la funcionalidad Blueprint de Render basada en Infrastructure as Code (render.yaml). Al conectar el repositorio, Render analizó el archivo descriptivo e instanció de manera coordinada la base de datos gestionada agrosmart-postgres y el servicio web agrosmart-n8n.

Figura 3

Conexión del repositorio en Render para inicializar el Blueprint



Nota. Captura de pantalla del catálogo de repositorios en Render mostrando la selección de Agrosmart-Insights. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 4

Configuración de variables de entorno y API Keys en el Blueprint de Render

You are deploying from a Blueprint for **equipo2-AgroSmart-Insights/Agrosmart-Insights**

Blueprint Name
A unique name for your Blueprint

Branch
The repository branch with the `render.yaml` file

Blueprint Path (Optional)
The path to the Blueprint file in your repo (e.g., `infra/render.yaml`). Defaults to `render.yaml` at the root of your repository.

Review Blueprint configurations from **equipo2-AgroSmart-Insights/Agrosmart-Insights**.

All future updates to your Blueprint file will be synced automatically, which may change your costs.

Specified configurations

- Create database **agrosmart-postgres**
- Create web service **agrosmart-n8n**

Key	Value
DB_HOST	Enter value
DB_NAME	Enter value
DB_USER	Enter value
DB_PASS	Enter value
DB_PORT	Enter value
DB_SCHEMA	Enter value
DB_ENGINE	Enter value
DB_COLLATION	Enter value

Nota. Captura de pantalla del formulario de Blueprint en Render con las variables del stack IA (Groq, Gemini, Hugging Face, MapTiler) ingresadas de forma segura. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 5

Confirmación y creación de servicios agrosmart-postgres y agrosmart-n8n

Create a new Blueprint Instance in Gabo's Workspace

Connect your Git repository or use an existing public repository URL.

Connect a repository

- equipo2-AgroSmart-Insights / Agrosmart-Insights** - 2 minutes ago
- Gabou.C / skills-introduction-to-github** - 12 days ago
- Gabou.C / equipo-2-AgroSmart-Insights** - 17 days ago
- Aguyamirto / Control_note** - 2 months ago
- Aguyamirto / TrabajaIA** - 2 months ago
- Gabou.C / Control_noteGabo** - 3 months ago

GitHub

- @Gabou.C** - 29 repos
- @Aguyamirto** - 2 repos
- @FranciscoDelgado** - 1 repo
- @equipo2-AgroSmart-Insights** - 1 repo

Configure account

GitLab

Connect account

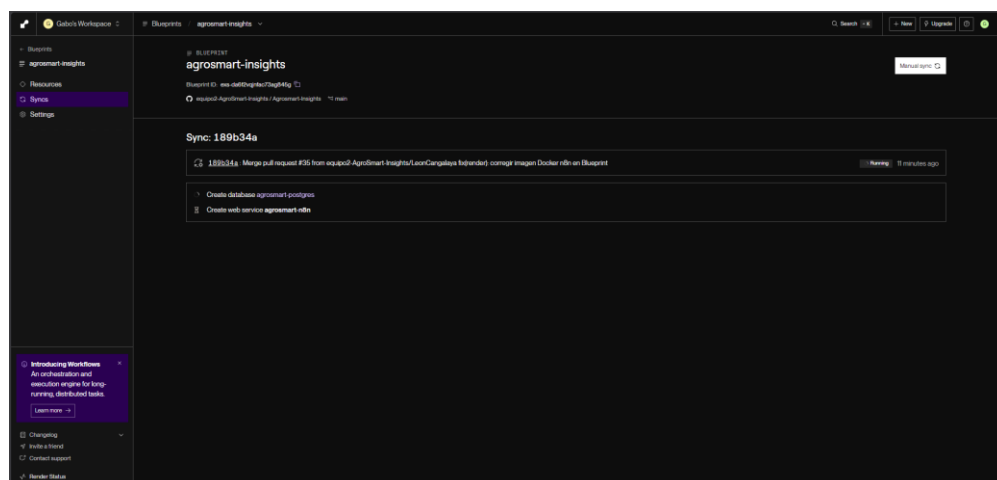
Bitbucket

Connect account

Nota. Captura de pantalla del resumen de recursos definidos en `render.yaml` antes de iniciar el aprovisionamiento en la nube. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 6

Sincronización y despliegue simultáneo de recursos en Render



Nota. Captura de pantalla mostrando el estado Running de la sincronización del Blueprint agrosmart-insights. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

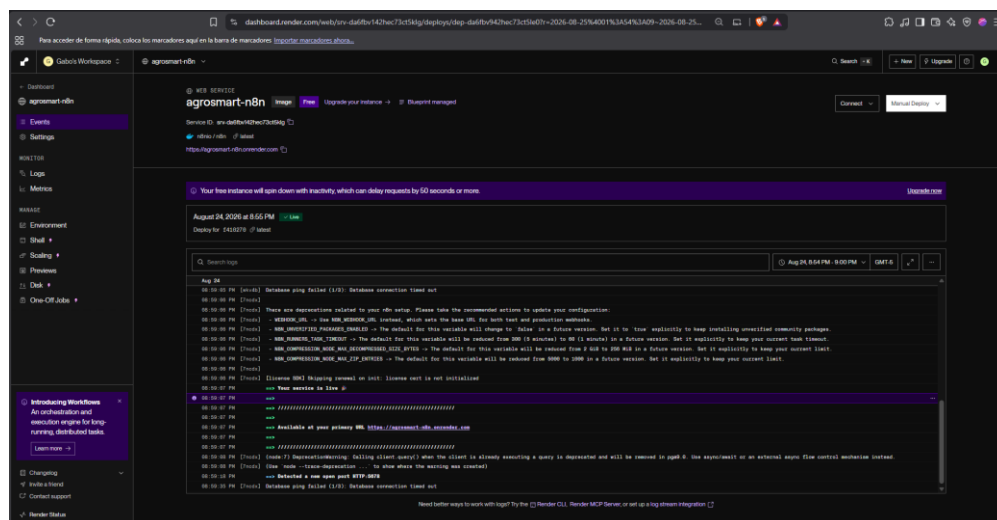
5. Diagnóstico y Resolución de Incidencias Técnicas

5.1. Incidencia de memoria insuficiente (OOM) en instancia n8n

Tras el despliegue inicial, el servicio agrosmart-n8n respondía con Not Found y se reiniciaba periódicamente. El análisis de logs reveló la excepción FATAL ERROR: JavaScript heap out of memory (código de salida 134), provocada por el consumo de memoria de Node.js al superar el límite de 512 MB del tier gratuito de Render.

Figura 7

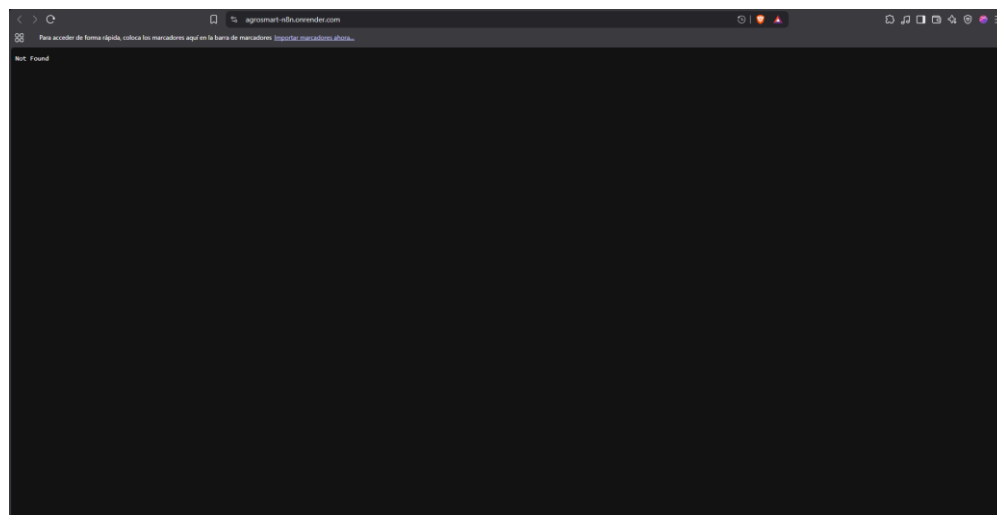
Logs de Render mostrando arranque inicial y advertencias del servicio



Nota. Captura de pantalla del visor de registros de Render con el mensaje n8n ready on port 5678 antes de registrar la saturación de heap. *Fuente:* Elaboración propia a partir de los logs de Render (Render, 2026).

Figura 8

Fallo de disponibilidad (Not Found) en la URL pública de n8n



Nota. Captura de pantalla del navegador web reflejando la indisponibilidad temporal de la interfaz de n8n debido a los reinicios continuos por OOM. *Fuente:* Elaboración propia a partir de la navegación web.

Se resolvieron los fallos mediante las siguientes optimizaciones:

- Fijación de versión: Se migró de :latest a la imagen estable `docker.io/n8nio/n8n:1.83.2`.
- Límite estricto de Heap: Se inyectó `NODE_OPTIONS=--max-old-space-size=384`, restringiendo el recolector de basura de Node.js a 384 MB.
- Desactivación de Runners: Se configuró `N8N_RUNNERS_ENABLED=false` para eliminar procesos auxiliares concurrentes.
- Alineación de puerto y URL: Se definieron explícitamente `PORT=5678` y `N8N_EDITOR_BASE_URL`.

5.2. Incidencia de inconsistencia en esquema interno de n8n (User.role)

Al acceder al asistente de bienvenida `/setup`, se produjo el error `column User.role does not exist` debido a que los reinicios por falta de memoria interrumpieron las migraciones internas automáticas de TypeORM en PostgreSQL.

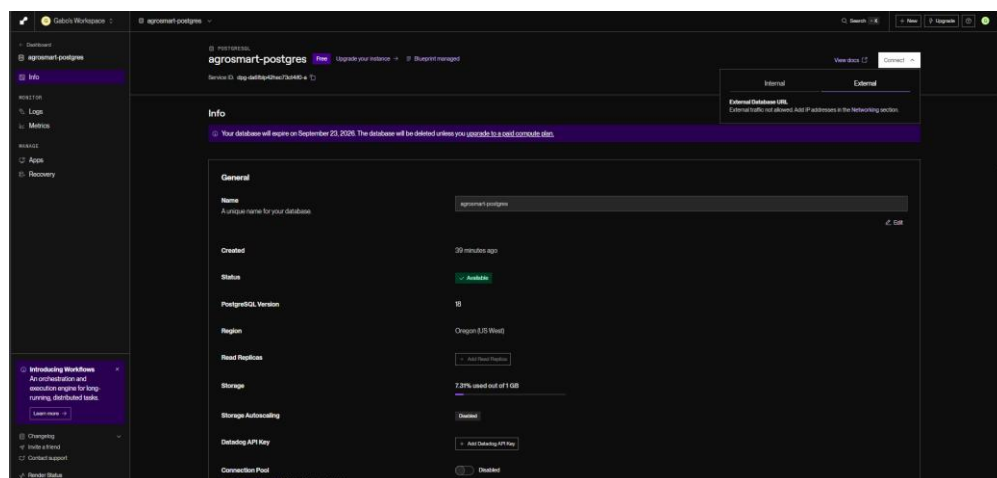
Figura 9

Panel de conexión y parámetros de la base de datos agrosmart-postgres

Nota. Captura de pantalla de la base de datos agrosmart-postgres en estado Available, detallando credenciales y puertos de acceso. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 10

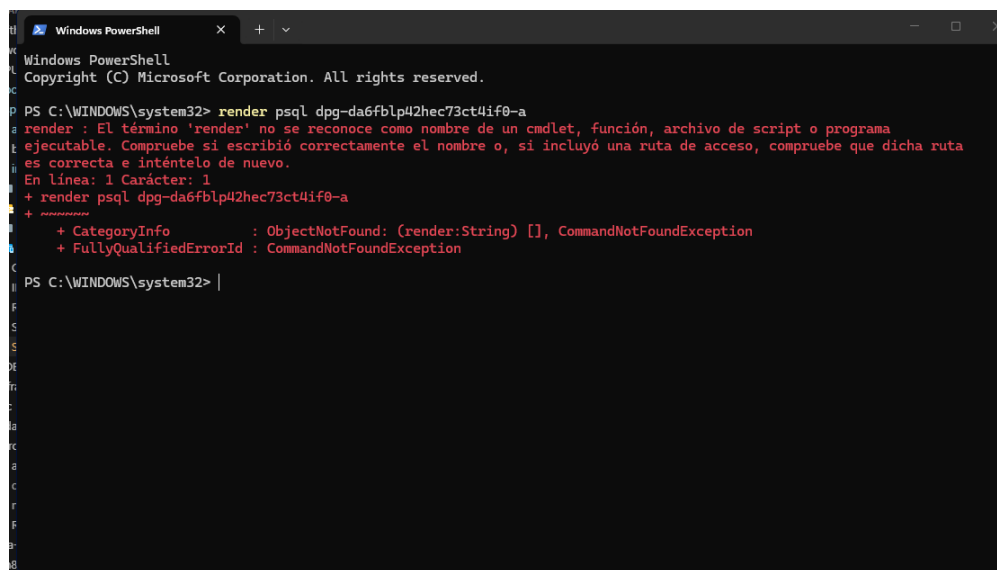
Habilitación de reglas de acceso IP en PostgreSQL Inbound Rules



Nota. Captura de pantalla de la sección Networking de la base de datos permitiendo el tráfico entrante desde la estación de trabajo del Líder DevSecOps. *Fuente:* Elaboración propia a partir de la consola de Render (Render, 2026).

Figura 11

Notificación de error en n8n: column User.role does not exist



Nota. Captura de pantalla del error al registrar la cuenta principal en /setup como consecuencia del esquema corrupto. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

Para restablecer la integridad:

- Se autorizó la IP pública de desarrollo en PostgreSQL Inbound IP Rules.

- Se ejecutó mediante psql la sentencia DROP SCHEMA public CASCADE; CREATE SCHEMA public; GRANT ALL ON SCHEMA public TO agrosmart;.
- Se reinició n8n con Manual Deploy, permitiendo que completara sus 25 migraciones limpias sobre PostgreSQL 18.

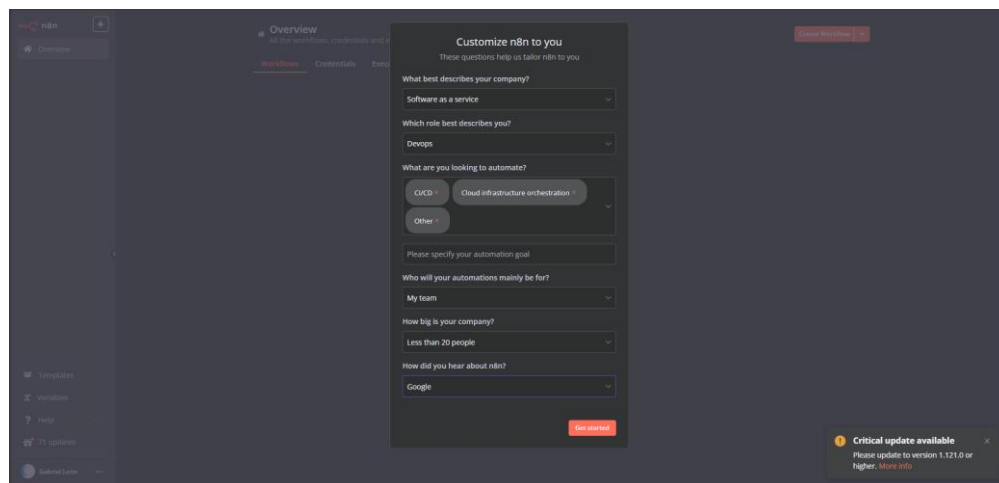
6. Validación de Servicios Operativos y Carga de Datos

6.1. Acceso a n8n y configuración de cuenta Owner

El servicio n8n quedó 100% operativo en <https://agrosmart-n8n.onrender.com>. Se configuró exitosamente la cuenta administrativa a nombre de Gabriel Emilio León Cangalaya y se verificó el estado saludable del motor.

Figura 12

Dashboard principal de n8n con cuenta Owner configurada y activa



Nota. Captura de pantalla del panel operativo de n8n mostrando la sesión iniciada del usuario Gabriel León y el editor listo para ejecución de flujos. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

6.2. Ejecución de migraciones de negocio y carga masiva SQL

Desde la consola PowerShell local y mediante el cliente psql, se ejecutaron los scripts de base de datos sobre la instancia cloud de PostgreSQL:

- 001_schema.sql: Creación de extensiones vector, tablas productos, mercados, precios_historicos y bitácoras de auditoría.
- 002_indexes.sql: Creación de índices compuestos para optimización de consultas.
- 002_seed_v2.sql: Inserción masiva de 9 445 filas de precios históricos oficiales del Gran Mercado Mayorista de Lima (GMML).

Figura 13

Confirmación de ejecución de migraciones y carga de 9 445 registros en PostgreSQL

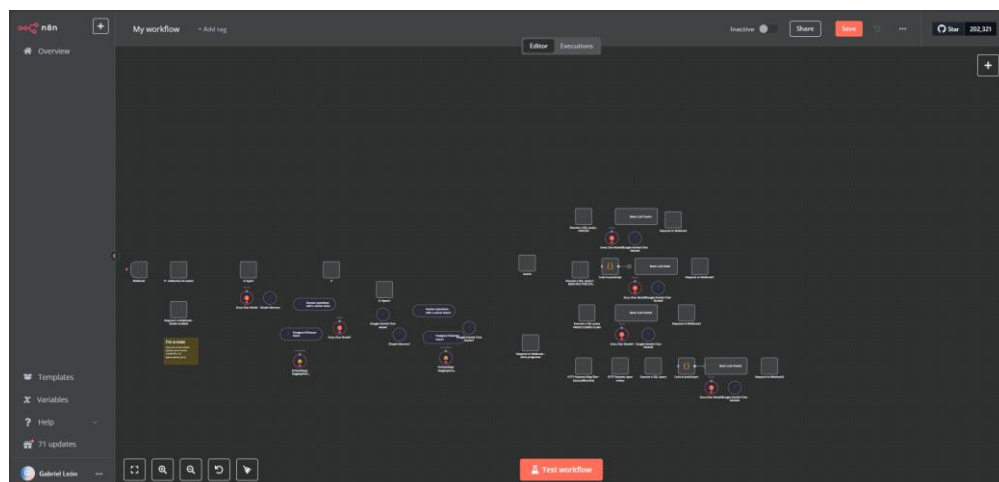
Nota. Captura de pantalla de la terminal psql mostrando las respuestas CREATE EXTENSION, CREATE TABLE, CREATE INDEX y COPY 9445. *Fuente:* Elaboración propia a partir de la consola de psql.

6.3. Importación, compatibilidad y activación del flujo WF2

Se importó wf2-api-chat-analisis-predictivo.json y se configuraron las credenciales Groq, Gemini, Hugging Face y PostgreSQL. El primer intento de activación falló por typeVersion posteriores a n8n 1.83.2 y por el campo Data Name vacío en las herramientas de vector store. Se corrigió el JSON en el repositorio, se reimportó el flujo y se verificó el webhook de producción.

Figura 14

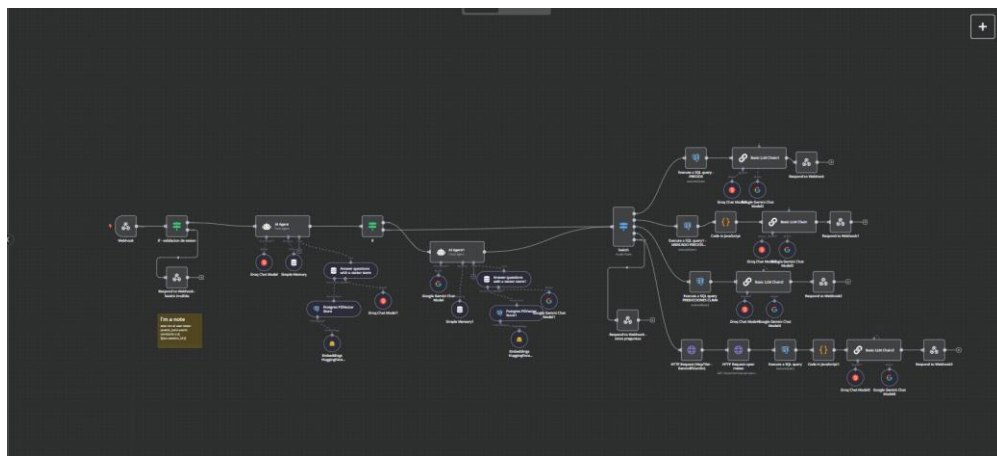
Workflow WF2 importado en el editor visual de n8n



Nota. Captura del lienzo de n8n con el grafo de nodos del flujo WF2 tras la importación. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

Figura 15

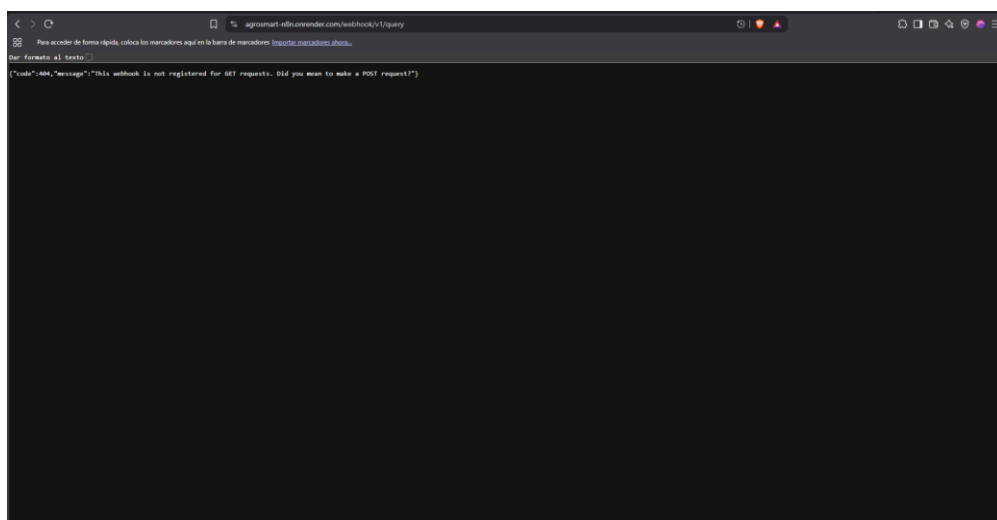
Flujo WF2 operativo con agentes Groq/Gemini, RAG y ramas SQL



Nota. Captura del lienzo completo de WF2 ya enlazado a credenciales y listo para recibir POST en /webhook/v1/query. *Fuente:* Elaboración propia a partir de la interfaz de n8n.

Figura 16

Confirmación pública de que el webhook WF2 está registrado y espera POST



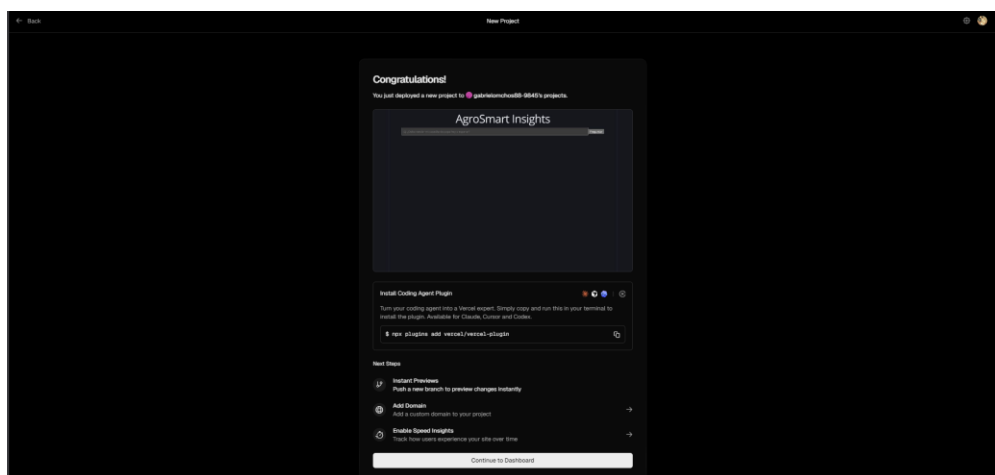
Nota. Captura del navegador en la URL de producción. El 404 de método GET demuestra que el disparador está activo. *Fuente:* Elaboración propia a partir de la navegación web.

7. Despliegue del frontend y prueba de extremo a extremo

El 25 de agosto de 2026 se publicó agrosmart-insights en Vercel (Hobby), con VITE_N8N_WEBHOOK_URL apuntando al webhook de Render. La primera consulta en el chat («¿Cuál es el precio actual de la papa?») evidenció que el cliente abortaba a los 6 segundos, mientras WF2 en Render continúa procesando. Se incrementó TIMEOUT_MS a 30 000 ms en n8nClient.js para el Pull Request de cierre.

Figura 17

Despliegue exitoso del frontend en Vercel



Nota. Captura de la pantalla Congratulations de Vercel con la vista previa de AgroSmart Insights. *Fuente:* Elaboración propia a partir de la consola de Vercel (Vercel, 2026).

Figura 18

Timeout de 6 segundos en la primera prueba E2E del chat en Vercel



Nota. Captura de la interfaz publicada. El mensaje en rojo corresponde al AbortController del cliente, no a una caída de n8n. *Fuente:* Elaboración propia a partir de la aplicación desplegada en Vercel.

8. Conclusión

En las sesiones del 24 y 25 de agosto de 2026 se cerró el despliegue cloud del Sprint 1 bajo el rol de Líder DevSecOps: main protegida, frontend en Vercel, n8n y PostgreSQL en Render, 9 445 registros cargados, WF2 compatible con n8n 1.83.2 y webhook público verificado. El único cambio de código pendiente de fusión es el timeout del cliente NLQ (y la compatibilidad de typeVersion de WF2), que habilita la prueba E2E completa tras la aprobación del arquitecto.

Referencias

Equipo 2 AgroSmart Insights. (2026). Agrosmart-Insights [Repositorio de software].

GitHub. <https://github.com/equipo2-AgroSmart-Insights/Agrosmart-Insights>

GitHub. (s. f.-a). Acerca de las ramas protegidas. Documentación de GitHub.

<https://docs.github.com/es/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches>

GitHub. (s. f.-b). Secretos en GitHub Actions. Documentación de GitHub.

<https://docs.github.com/es/actions/concepts/security-for-github-actions/security-guides/using-secrets-in-github-actions>

National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1 (NIST SP 800-218).

<https://csrc.nist.gov/Projects/ssdf>

Red Hat. (s. f.). ¿Qué es DevSecOps? <https://www.redhat.com/es/topics/devops/what-is-devsecops>

Render. (2026). Blueprint specification. <https://render.com/docs/blueprint-spec>

Render. (2026). Deploy n8n on Render. <https://render.com/docs/deploy-n8n>

Vercel. (2026). Deploying a Git repository. <https://vercel.com/docs/deployments/git>

Vercel. (2026). Monorepos. <https://vercel.com/docs/monorepos>